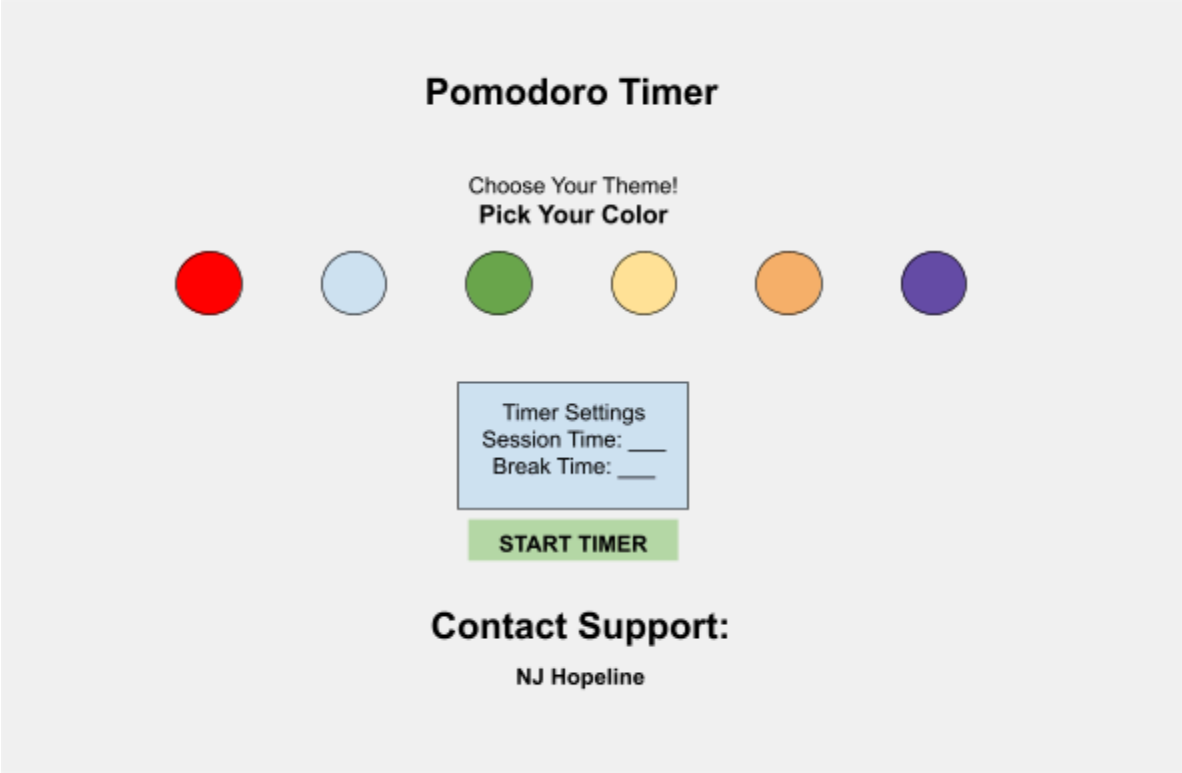
User Interface Design





Modularity and Encapsulation

The timer controller handles only the logic for processing inputs and managing timer parameters, while the views focus exclusively on presenting the interface to users. This separation allows either component to be modified independently.

The timer functionality organizes code into functions like startTimer(), pauseTimer(), and updateTimer(). State variables such as minutes, seconds, and isRunning, hiding functionality from other parts of the application.

Algorithm Elegance

The timer implementation uses an elegant and efficient approach to time tracking. The timing system uses javascript intervals, eliminating ruby requirement for constant server communication updating pages every second. This keeps the application responsive.

The state transition logic between work and break sessions is elegant, using a simple boolean toggle (inSession = !inSession). The countdown algorithm efficiently handles countdown displays and zero detection with relatively simple code.

Appropriate Data Structures

Application uses basic integer values for time tracking and boolean flags for state management. Primitive types provide everything needed without unnecessary complexity.

The application avoids using a database since it doesn't require persistence across sessions. Timer interval ID is properly managed in a dedicated variable. This straightforward approach to data management keeps the application fast and responsive.